CUDA Memory Hierarchy - 성능의 핵심

학습 목표

- CUDA의 여러 메모리 종류 이해 (Global, Shared, Register, etc.)
- **Memory Coalescing** - 성능 차이 100배!
- Cache 활용 전략
- 실전 메모리 최적화 기법

예상 학습 시간: 4-5시간
난이도: ⭐⭐ (초급-중급)
선수 과목: 00_CUDA_Introduction.md, 01_CUDA_Programming_Model.md

---

1. Memory Hierarchy 개요

1.1 속도와 용량의 트레이드오프

```
┌──────────────────────────────────────────────────────────┐
│  속도  ↑                                    용량  ↓  │
├──────────────────────────────────────────────────────────┤
│  Register (per-thread)                                   │
│  - Latency: 1 cycle                                      │
│  - Size: ~64 KB per SM (256 KB / 4 warp schedulers)      │
│  - Scope: Thread private                                 │
├──────────────────────────────────────────────────────────┤
│  Shared Memory (per-block)                               │
│  - Latency: ~30 cycles                                   │
│  - Size: 128 KB per SM (configurable)                    │
│  - Scope: Block-wide shared                              │
├──────────────────────────────────────────────────────────┤
│  L1 Cache (per-SM)                                       │
│  - Latency: ~80 cycles                                   │
│  - Size: ~128 KB (shared with smem)                      │
│  - Scope: Automatic                                      │
```

```
├──────────────────────────────────────────────────────────┤
│  L2 Cache (GPU-wide)                                     │
│  - Latency: ~200 cycles                                  │
│  - Size: 40-80 MB (GPU dependent)                        │
│  - Scope: All SMs                                        │
├──────────────────────────────────────────────────────────┤
│  Global Memory (Device DRAM)                             │
│  - Latency: ~400-800 cycles                              │
│  - Size: 12-96 GB (GPU dependent)                        │
│  - Bandwidth: 200-1000 GB/s                              │
├──────────────────────────────────────────────────────────┤
│  Host Memory (System RAM)                                │
│  - Latency: ~100,000+ cycles (PCIe transfer)             │
│  - Size: 16-256 GB+                                      │
│  - Bandwidth: 12-64 GB/s (PCIe 4.0/5.0 x16)              │
└──────────────────────────────────────────────────────────┘
  속도  ↓                                        용량  ↑
```

1.2 성능 비교 (실측치)

```
// Benchmark: 1M 덧셈 연산
Register:       0.01 ms  (baseline)
Shared Memory:  0.3 ms   (30배 느림)
L1 Cache:       0.8 ms   (80배 느림)
Global (coalesced): 4 ms (400배 느림)
Global (uncoalesced): 40 ms (4000배 느림!) ⚠️
Host Memory:    1000 ms  (100,000배 느림!)
```

교훈:

- Register 최대한 활용!
- Global memory 접근 최소화
- **Coalescing은 필수!** (10배 차이)

---

2. Register Memory

2.1 특징

- **가장 빠른 메모리** (1 cycle latency)

- Thread마다 **private** (다른 threads 접근 불가)
- SM당 **65,536개 32-bit registers** (Ampere/Ada)
- **자동 할당** (변수 선언 시)

```
__global__ void example() {
    int a = 5;           // Register
    float b = 3.14f;     // Register
    double c = 2.718;    // 2 registers (64-bit)

    // 계산도 register에서
    float result = a * b + c;  // 모두 register 연산!
}
```

2.2 Register Spilling (피해야 함!)

```
// 문제: 레지스터 너무 많이 사용
__global__ void heavy_kernel() {
    float arr[100];  // ❌ 100개 registers 필요 → Spilling!
    for (int i = 0; i < 100; i++) {
        arr[i] = i * i;
    }
    // ...
}

// Spilling 발생 시:
// - Register → Local Memory (Global memory의 일부!)
// - 성능 급격히 저하 (1 cycle → 400 cycles)
```

확인 방법:

```
nvcc --ptxas-options=-v kernel.cu
# 출력:
# ptxas info : Used 48 registers, 128 bytes lmem, 1024 bytes smem
#                                 ^^^^^ Local memory (spilling!)
```

해결 방법:

```
// 방법 1: 배열을 shared memory로
__global__ void optimized_kernel() {
    __shared__ float arr[100];
    // ...
}
```

```
// 방법 2: 레지스터 제한
// nvcc -maxrregcount=32 kernel.cu

// 방법 3: 알고리즘 재설계 (streaming 방식)
```

2.3 Register Pressure와 Occupancy

```
// SM당 레지스터: 65,536개
// Thread당 레지스터 사용량 → Occupancy 제한

// Case 1: 32 registers/thread
// Max threads = 65,536 / 32 = 2048 threads per SM
// → 100% occupancy 가능 (max 2048)

// Case 2: 64 registers/thread
// Max threads = 65,536 / 64 = 1024 threads per SM
// → 50% occupancy

// Case 3: 128 registers/thread
// Max threads = 65,536 / 128 = 512 threads per SM
// → 25% occupancy
```

---

3. Global Memory

3.1 특징

- **가장 큰 메모리** (8-96 GB)
- **가장 느림** (400-800 cycles latency)
- 모든 threads 접근 가능
- `cudaMalloc()` 으로 할당

```
// Global memory 할당
float* d_data;
cudaMalloc(&d_data, N * sizeof(float));

__global__ void kernel(float* data) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
```

```
    data[tid] = 0.0f;  // Global memory 쓰기
}
```

3.2 Memory Coalescing - 초중요! 🔥

(1) Coalesced Access (Good)

```
__global__ void coalesced_read(float* data) {
    int tid = threadIdx.x;  // 0, 1, 2, 3, ..., 31 (warp)

    // Warp 내 threads가 연속된 메모리 접근
    float value = data[tid];
    // Thread 0 → data[0]
    // Thread 1 → data[1]
    // Thread 2 → data[2]
    // ...
    // Thread 31 → data[31]

    // → 단일 128-byte 메모리 트랜잭션! ✅
}

메모리 트랜잭션:
┌──────────────────────────────────────────┐
| data[0-31] - 128 bytes (32 floats × 4 bytes)|  ← 1번 트랜잭션
└──────────────────────────────────────────┘
성능: 100%
```

(2) Uncoalesced Access (Bad)

```
__global__ void uncoalesced_read(float* data) {
    int tid = threadIdx.x;  // 0, 1, 2, ..., 31

    // Stride access (띄엄띄엄 접근)
    float value = data[tid * 32];
    // Thread 0  → data[0]
    // Thread 1  → data[32]
    // Thread 2  → data[64]
    // ...
    // Thread 31 → data[992]

    // → 32번의 메모리 트랜잭션! ❌ (32배 느림!)
}
```

```
메모리 트랜잭션:
┌──────┐        ┌──────┐        ┌──────┐
|data[0]| ... |data[32]| ... |data[64]| ... (32개 트랜잭션!)
└──────┘        └──────┘        └──────┘
성능: 3% (32배 느림)
```

(3) 실험: AoS vs SoA Layout

Array of Structures (AoS) - ❌ Uncoalesced:

```
struct Particle {
    float x, y, z;     // 위치
    float vx, vy, vz;  // 속도
};

Particle* particles;  // [p0, p1, p2, ...]

__global__ void update_aos(Particle* p, int N) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    if (tid < N) {
        // Warp가 x 접근 시:
        // Thread 0: p[0].x
        // Thread 1: p[1].x  ← p[0]과 24 bytes 떨어짐!
        // → Uncoalesced (stride = 24 bytes)
        p[tid].x += p[tid].vx * dt;
    }
}

메모리 레이아웃:
[x0 y0 z0 vx0 vy0 vz0][x1 y1 z1 vx1 vy1 vz1]...
 ^                     ^
 Thread 0              Thread 1 (24 bytes 떨어짐)
```

Structure of Arrays (SoA) - ✅ Coalesced:

```
struct ParticlesSoA {
    float* x;   // 모든 x 좌표
    float* y;   // 모든 y 좌표
    float* z;
    float* vx;
    float* vy;
    float* vz;
};
```

```
__global__ void update_soa(float* x, float* vx, int N, float dt) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    if (tid < N) {
        // Warp가 x 접근 시:
        // Thread 0: x[0]
        // Thread 1: x[1]  ← x[0]과 4 bytes 떨어짐 (연속!)
        // → Coalesced!
        x[tid] += vx[tid] * dt;
    }
}

메모리 레이아웃:
x: [x0][x1][x2][x3]... (연속!)
    ^   ^
    T0  T1 (4 bytes, coalesced!)
```

벤치마크 결과 (1M particles):

```
AoS: 45 ms
SoA: 5 ms
Speedup: 9x! 🚀
```

3.3 Alignment와 Padding

128-byte aligned access가 최적:

```
// ✅ Good - aligned
float* data;
cudaMalloc(&data, N * sizeof(float));  // cudaMalloc은 자동 align

__global__ void aligned_access(float* data) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    data[tid] = 1.0f;  // Coalesced + aligned
}

// ❌ Bad - misaligned offset
__global__ void misaligned_access(float* data) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    data[tid + 7] = 1.0f;  // +7 offset → misalignment
    // Warp 0: data[7], data[8], ..., data[38]
    // → 2번 트랜잭션 (data[0-31], data[32-63])
}
```

Struct padding 예시:

```
// ❌ Bad layout
struct Bad {
    float a;   // 4 bytes
    double b;  // 8 bytes - misaligned!
    float c;   // 4 bytes
};  // Total: 20 bytes → padding to 24

// ✅ Good layout (aligned)
struct Good {
    double b;  // 8 bytes - aligned
    float a;   // 4 bytes
    float c;   // 4 bytes
};  // Total: 16 bytes, all aligned
```

---

4. Shared Memory

4.1 특징

- **Register 다음으로 빠름** (~30 cycles)
- **Block 내 threads 공유**
- SM당 **128 KB** (Ampere/Ada, configurable)
- `__shared__` 키워드로 선언

```
__global__ void use_shared_memory() {
    // Shared memory 선언
    __shared__ float tile[256];

    int tid = threadIdx.x;

    // 모든 threads가 접근 가능
    tile[tid] = tid * 2.0f;

    __syncthreads();  // 모든 threads가 쓰기 완료 대기

    // 다른 thread 데이터 읽기
    float neighbor = tile[(tid + 1) % 256];
}
```

4.2 Bank Conflicts (성능 저하!)

Shared memory는 **32개 banks**로 구성 (4-byte 단위):

```
Bank 구조:
Bank 0:  addr[0], addr[32], addr[64], ...
Bank 1:  addr[1], addr[33], addr[65], ...
Bank 2:  addr[2], addr[34], addr[66], ...
...
Bank 31: addr[31], addr[63], addr[95], ...

같은 bank에 동시 접근 → Conflict!
```

(1) No Conflict - ✅ Best

```
__shared__ float data[32];

__global__ void no_conflict() {
    int tid = threadIdx.x;  // 0-31 (warp)

    // 각 thread가 다른 bank 접근
    float val = data[tid];
    // Thread 0 → Bank 0
    // Thread 1 → Bank 1
    // ...
    // Thread 31 → Bank 31
    // → 1 cycle (conflict 없음!)
}
```

(2) 2-way Conflict - ❌ Bad

```
__shared__ float data[64];

__global__ void two_way_conflict() {
    int tid = threadIdx.x;  // 0-31

    // Stride-2 access
    float val = data[tid * 2];
    // Thread 0  → data[0]  → Bank 0
    // Thread 1  → data[2]  → Bank 2
    // ...
    // Thread 16 → data[32] → Bank 0 (Conflict with Thread 0!)
```

```
    // → 2 cycles (2배 느림)
}
```

(3) 32-way Conflict - ❌❌ Worst

```
__shared__ float data[1024];

__global__ void worst_conflict() {
    int tid = threadIdx.x;  // 0-31

    // 모든 threads가 같은 bank
    float val = data[tid / 32];
    // 모든 threads → data[0] → Bank 0
    // → 32 cycles (32배 느림!)
}
```

4.3 Bank Conflict 해결 전략

전략 1: Padding 추가

```
// ❌ Bad - conflict 발생
__shared__ float tile[32][32];  // Bank conflict!

__global__ void bad() {
    int tid = threadIdx.x;
    // tile[0][0], tile[0][1], ..., tile[0][31]
    // → Bank 0, Bank 1, ..., Bank 31
    // tile[1][0], tile[1][1], ...
    // → Bank 0, Bank 1, ... (같은 bank!)
    float val = tile[tid][0];  // Column access → conflict!
}

// ✅ Good - padding으로 해결
__shared__ float tile[32][33];  // +1 padding

__global__ void good() {
    int tid = threadIdx.x;
    // tile[0][0-32]: Bank 0-32 (Bank 32는 Bank 0과 다름!)
    // tile[1][0-32]: Bank 1-33 (shifted!)
    float val = tile[tid][0];  // No conflict!
}
```

메커니즘:

```
Without padding [32][32]:
Row 0: Bank 0,  1,  2, ..., 31
Row 1: Bank 0,  1,  2, ..., 31 ← Same banks!

With padding [32][33]:
Row 0: Bank 0,  1,  2, ..., 31, (32)
Row 1: Bank 1,  2,  3, ..., 32,  0  ← Shifted!
Row 2: Bank 2,  3,  4, ...,  0,  1  ← Different banks
```

전략 2: 메모리 액세스 패턴 변경

```
// Transpose 연산
__shared__ float tile[32][32 + 1];  // Padding

__global__ void transpose() {
    int x = threadIdx.x, y = threadIdx.y;

    // Load (coalesced + no conflict)
    tile[y][x] = input[y * width + x];

    __syncthreads();

    // Store (transposed, but still no conflict due to padding)
    output[x * width + y] = tile[x][y];
}
```

4.4 Shared Memory vs L1 Cache 설정

```
// SM당 128 KB를 shared memory / L1 cache로 분할

// 방법 1: Device-wide 설정
cudaDeviceSetCacheConfig(cudaFuncCachePreferShared);
// - Shared: 100 KB, L1: 28 KB

cudaDeviceSetCacheConfig(cudaFuncCachePreferL1);
// - Shared: 28 KB, L1: 100 KB

cudaDeviceSetCacheConfig(cudaFuncCachePreferEqual);
// - Shared: 64 KB, L1: 64 KB
```

```
// 방법 2: Kernel별 설정
cudaFuncSetCacheConfig(myKernel, cudaFuncCachePreferShared);
```

---

5. Constant Memory

5.1 특징

- **읽기 전용** (kernel 실행 중)
- **64 KB 제한**
- **Broadcast 최적화** (모든 threads가 같은 값 읽으면 빠름)
- Cache됨 (8 KB per SM)

```
// Host 코드
__constant__ float const_data[1024];  // Constant memory

int main() {
    float h_data[1024];
    // ... 초기화 ...

    // Host → Constant memory 복사
    cudaMemcpyToSymbol(const_data, h_data, sizeof(h_data));

    kernel<<<grid, block>>>();
}

// Kernel
__global__ void kernel() {
    int tid = threadIdx.x;

    // Constant memory 읽기 (빠름!)
    float coeff = const_data[5];  // 모든 threads가 같은 값 읽음
    // → Single broadcast (1 cycle)

    // 하지만...
    float val = const_data[tid];  // 각 thread가 다른 값 읽음
    // → 32 reads (warp divergence처럼 serialize됨)
}
```

5.2 사용 사례

```
// ✅ Good use case: 필터 계수
__constant__ float gaussian_kernel[25] = {
    1,  4,  6,  4, 1,
    4, 16, 24, 16, 4,
    6, 24, 36, 24, 6,
    4, 16, 24, 16, 4,
    1,  4,  6,  4, 1
};

__global__ void gaussian_blur(float* image, float* output, int width) {
    int x = threadIdx.x + blockIdx.x * blockDim.x;
    int y = threadIdx.y + blockIdx.y * blockDim.y;

    float sum = 0.0f;
    for (int ky = -2; ky <= 2; ky++) {
        for (int kx = -2; kx <= 2; kx++) {
            // 모든 threads가 같은 순서로 읽음 → cached & broadcasted
            sum += image[(y+ky)*width + (x+kx)] * gaussian_kernel[(ky+2)*5 + (kx
        }
    }
    output[y * width + x] = sum / 256.0f;
}
```

---

6. Texture Memory (고급)

6.1 특징

- **읽기 전용**
- **2D spatial locality 최적화** (이미지 처리에 최적)
- **Hardware interpolation** 지원
- **Boundary handling** 자동

```
// Texture object 생성
cudaTextureObject_t create_texture(float* d_data, int width, int height) {
    cudaResourceDesc resDesc;
    memset(&resDesc, 0, sizeof(resDesc));
    resDesc.resType = cudaResourceTypePitch2D;
    resDesc.res.pitch2D.devPtr = d_data;
    resDesc.res.pitch2D.width = width;
```

```
    resDesc.res.pitch2D.height = height;
    resDesc.res.pitch2D.pitchInBytes = width * sizeof(float);
    resDesc.res.pitch2D.desc = cudaCreateChannelDesc<float>();

    cudaTextureDesc texDesc;
    memset(&texDesc, 0, sizeof(texDesc));
    texDesc.readMode = cudaReadModeElementType;

    cudaTextureObject_t tex;
    cudaCreateTextureObject(&tex, &resDesc, &texDesc, NULL);
    return tex;
}

// Kernel
__global__ void use_texture(cudaTextureObject_t tex, float* output, int width, i
    int x = threadIdx.x + blockIdx.x * blockDim.x;
    int y = threadIdx.y + blockIdx.y * blockDim.y;

    if (x < width && y < height) {
        // Hardware interpolation (bilinear)
        float val = tex2D<float>(tex, x + 0.5f, y + 0.5f);
        output[y * width + x] = val;
    }
}
```

---

7. 실습: Matrix Multiplication (Tiled)

7.1 Naive Version (Global memory만)

```
__global__ void matmul_naive(float* A, float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; k++) {
            // Global memory 접근: 2 * N번 (A, B 각각)
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
```

```
    }
}

// 성능 (N=1024): ~50 GFLOPS (이론치의 2%)
// Memory bandwidth bound!
```

7.2 Tiled Version (Shared memory 활용)

```
#define TILE_SIZE 16

__global__ void matmul_tiled(float* A, float* B, float* C, int N) {
    __shared__ float As[TILE_SIZE][TILE_SIZE];
    __shared__ float Bs[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;

    float sum = 0.0f;

    // Tile별로 처리
    for (int t = 0; t < (N + TILE_SIZE - 1) / TILE_SIZE; t++) {
        // Load tile into shared memory (coalesced!)
        if (row < N && t * TILE_SIZE + threadIdx.x < N)
            As[threadIdx.y][threadIdx.x] = A[row * N + t * TILE_SIZE + threadIdx
        else
            As[threadIdx.y][threadIdx.x] = 0.0f;

        if (col < N && t * TILE_SIZE + threadIdx.y < N)
            Bs[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE + threadIdx.y) * N +
        else
            Bs[threadIdx.y][threadIdx.x] = 0.0f;

        __syncthreads();

        // Compute using shared memory (fast!)
        for (int k = 0; k < TILE_SIZE; k++) {
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        }

        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
```

```
    }
}

// 성능 (N=1024): ~800 GFLOPS (이론치의 30%)
// 16배 향상! 🚀
```

왜 빠른가?

```
Naive:
- Global memory 접근: 2N번 per element
- Bandwidth: 2N * 4 bytes = 8N bytes

Tiled:
- Global memory: Tile 로드 시 2번만
- Shared memory: TILE_SIZE번 재사용
- Bandwidth: 2 * TILE_SIZE * 4 bytes (1/N 감소!)
```

---

8. 메모리 최적화 체크리스트

8.1 Global Memory 최적화

```
// ✅ DO:
// 1. Coalesced access (SoA layout)
struct ParticlesGood {
    float* x, *y, *z;  // SoA
};

// 2. Aligned allocation
cudaMalloc(&ptr, size);  // 자동 aligned

// 3. Minimize transfers
// - 데이터를 GPU에 최대한 유지
// - Batch 처리

// ❌ DON'T:
// 1. Uncoalesced (AoS for parallel access)
struct ParticlesBad {
    float x, y, z;
} particles[N];  // AoS
```

```
// 2. Frequent CPU-GPU transfers
for (int i = 0; i < 1000; i++) {
    cudaMemcpy(d_data, h_data, size, H2D);  // ❌ 매 iteration마다
    kernel<<<...>>>(d_data);
    cudaMemcpy(h_data, d_data, size, D2H);
}
```

8.2 Shared Memory 최적화

```
// ✅ DO:
// 1. Padding for bank conflict prevention
__shared__ float tile[32][33];  // +1 padding

// 2. Reuse data
// - Load once, use multiple times

// 3. Appropriate size
// - Balance with occupancy

// ❌ DON'T:
// 1. Excessive usage
__shared__ float huge[128][128];  // 64 KB - occupancy 저하!

// 2. Forget __syncthreads()
tile[tid] = data[tid];
// __syncthreads(); 누락 → Race condition!
float val = tile[tid + 1];
```

---

9. 프로파일링

9.1 Nsight Compute

```
# Memory throughput 측정
ncu --metrics dram__bytes_read.sum,dram__bytes_write.sum ./program

# Coalescing 효율성
ncu --metrics l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum,\
              l1tex__t_requests_pipe_lsu_mem_global_op_ld.sum \
    ./program
```

```
# Coalescing Efficiency = Requests / Sectors (100% = perfect)
```

9.2 Memory Bandwidth 계산

```
// Theoretical peak (RTX 4090)
// - Memory clock: 10.5 Gbps
// - Bus width: 384-bit = 48 bytes
// - Peak: 10.5 * 48 = 504 GB/s (GDDR6X 양방향 → ×2 = 1008 GB/s)

// Achieved bandwidth
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
kernel<<<grid, block>>>(d_data, N);
cudaEventRecord(stop);

cudaEventSynchronize(stop);
float ms;
cudaEventElapsedTime(&ms, start, stop);

size_t bytes = N * sizeof(float) * 2;  // read + write
float bandwidth = (bytes / 1e9) / (ms / 1000.0f);  // GB/s
printf("Bandwidth: %.1f GB/s (%.1f%% of peak)\n",
       bandwidth, bandwidth / 1008.0f * 100);
```

---

10. 다음 단계

메모리 최적화를 마스터했으니, 이제 동기화와 고급 최적화로 넘어가세요:

- [03_Thread_Synchronization.md]
  - `__syncthreads()`, `__syncwarp()`
  - Atomic operations
  - Memory fence

실습 예제:

- `examples/cuda/02_matrix_multiply.cu` - Tiling 기법 마스터
- `examples/cuda/03_reduction.cu` - Shared memory + warp primitives

요약 체크리스트

- [ ] Memory hierarchy 이해 (Register > Shared > Global)
- [ ] **Coalescing 개념 완벽 이해** (가장 중요!)
- [ ] SoA vs AoS layout 차이 파악
- [ ] Bank conflict 원인과 해결법 숙지
- [ ] Shared memory tiling 기법 구현 가능
- [ ] Constant memory 적절한 사용 사례 파악
- [ ] Memory bandwidth 측정 및 분석 가능

핵심 개념:

- **Coalescing = 10-100배 성능 차이!**
- SoA layout 사용 (병렬 연산 시)
- Shared memory로 재사용 극대화
- Bank conflict 피하기 (padding 추가)